TECH BIZ SECURITY · SOC ANALYST INTERNSHIP · BATCH B-503

WEEK 4 · TASK 11

Ingesting Logs into Splunk & Basic Searches

Submitted by	**Oscar ALIDJINOU**
Student ID	**TBS-INT-2026-2398**
Program	**SOC Analyst Internship (6 weeks)**
Company	**Tech Biz Security**
Date of submission	**08-11-2026**

01 Objective

A SIEM is only useful once it holds data. In Task 10 I installed Splunk; this task was about feeding it. I sent two real logs from my Ubuntu virtual machine into Splunk, created failed SSH logins to look for, and then used the Search Processing Language (SPL) to find them, count them, group them by user and source, and view them over time.

02 Proof of Completion

Room / activity	Duration	Tasks	Status
TryHackMe: Introduction to SIEM	120 min	7	Completed, room progress 100%
Lab: ingesting two logs into Splunk	-	-	Done (section 04)
Lab: three searches on failed logins	-	-	Done (section 06)

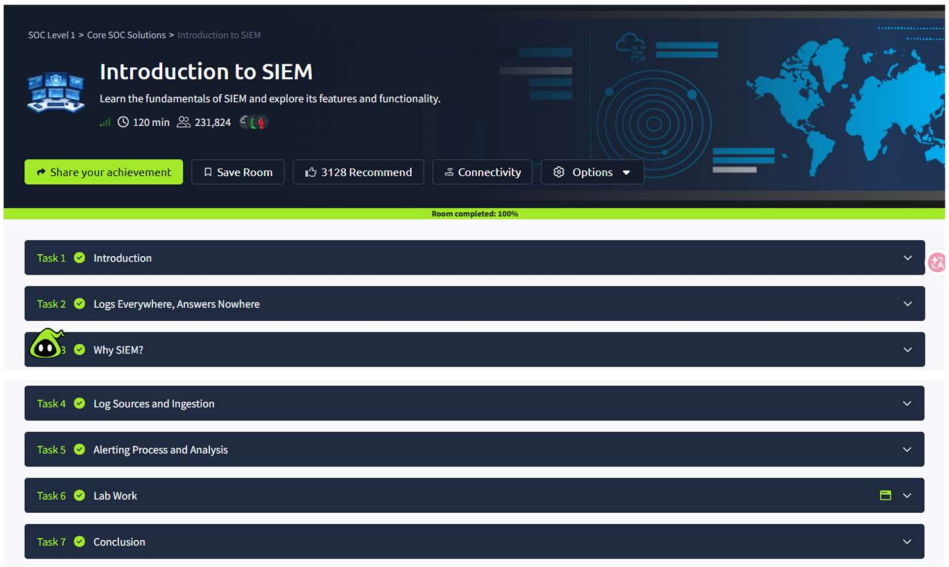


Figure 1. Introduction to SIEM: all 7 tasks completed (100%).

03 Key Concepts

3.1 Three ways to get data into Splunk

Method	What it does	When it fits
Upload	Sends a file from my computer once.	Quick analysis of an offline log file.
Monitor	Watches a file, a folder or a port on the Splunk machine and keeps indexing new events.	Logs on the same machine as Splunk, which is my lab case.
Forward	Receives data from a Splunk forwarder installed on another machine.	Collecting from many servers in a company.

3.2 Four fields that describe every event

Field	Meaning
host	The machine the event came from (here, ubuntu).
source	The file or input that produced it (here, /var/log/auth.log).
sourcetype	The type of data, which tells Splunk how to read its structure and timestamp.
index	The storage area where the event is kept. Mine went to the default index, main.

3.3 A little SPL

An SPL search starts with search terms, then passes the results through commands separated by the pipe symbol (|), each command working on the output of the one before. The time range picker next to the search bar limits which events are searched.

04 Ingestion Steps

Step 1: Check the log I want to ingest

I followed the authentication log live to see what it holds right now. It contained my own sudo activity, with a line when a session opens and another when it closes.

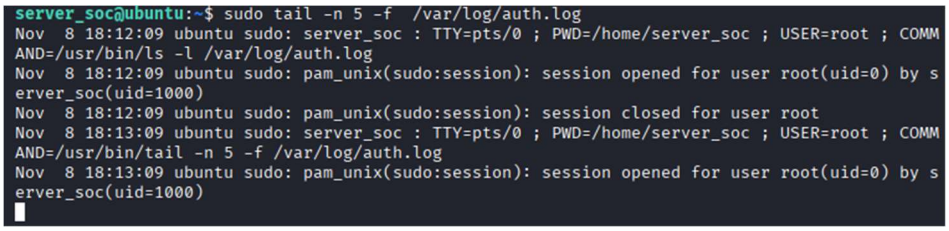


Figure 2. auth.log followed with tail: sudo session events.

Step 2: Give Splunk permission to read it

Splunk runs as its own user, splunk. The file /var/log/auth.log can only be read by root and by members of the adm group, so without a change Splunk would index nothing. I added the splunk user to the adm group and confirmed it with the id command. Splunk must be restarted for a new group membership to apply.

```
sudo usermod -aG adm splunk
id splunk
```

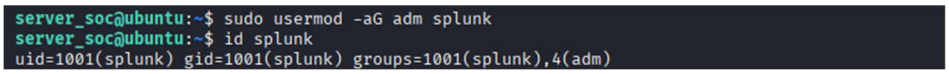


Figure 3. The splunk user now belongs to the adm group (group 4).

Step 3: Add the log file as a data input

In Splunk Web I opened Settings and chose Add Data. Of the three methods, I picked Monitor, then Files & Directories, because the log is on the same machine as Splunk, and then browsed to /var/log/auth.log.

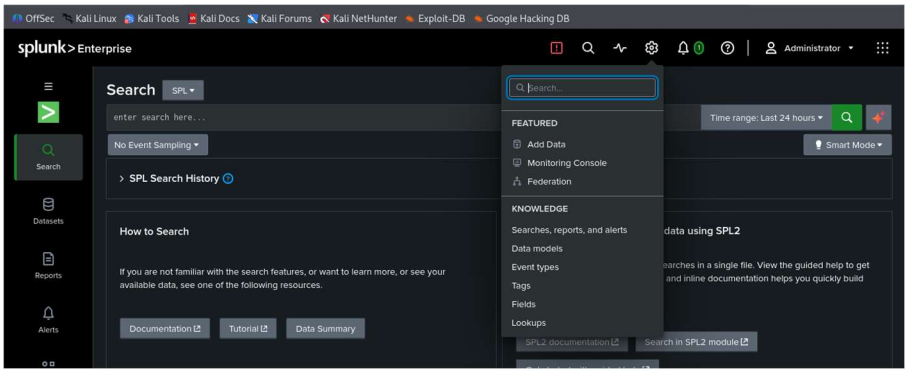


Figure 4. Settings menu: Add Data.

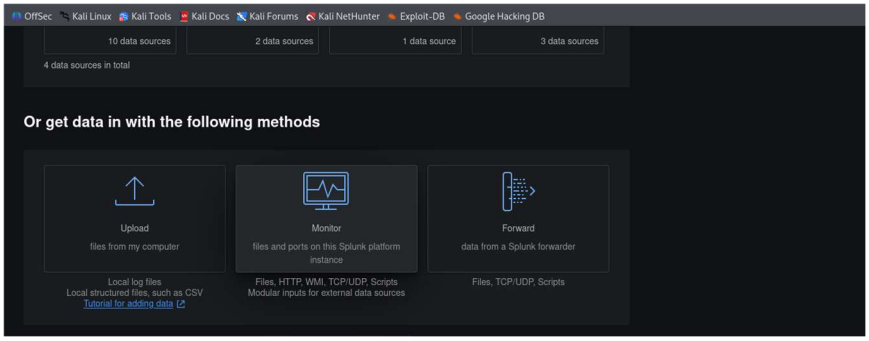


Figure 5. The three methods: Upload, Monitor and Forward.

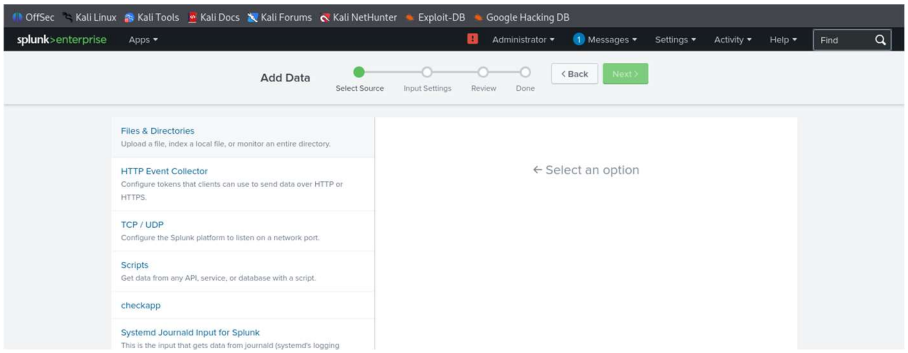


Figure 6. Monitor: Files & Directories.

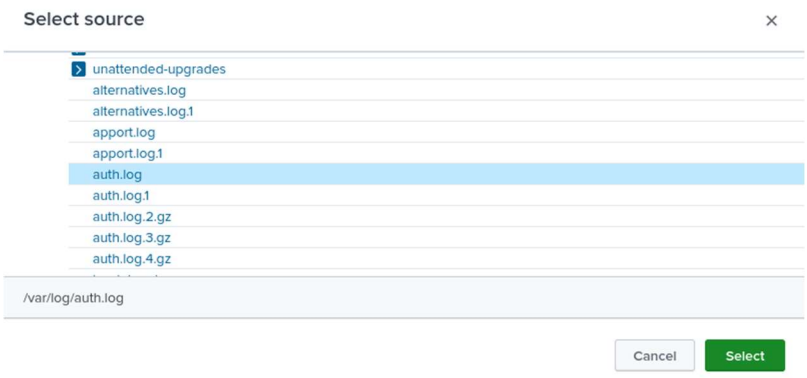


Figure 7. Selecting /var/log/auth.log.

The Set Source Type page previews how Splunk will read the events and their timestamps before indexing. The events looked correct, with a time and message on every line, so I continued to the review step and submitted.

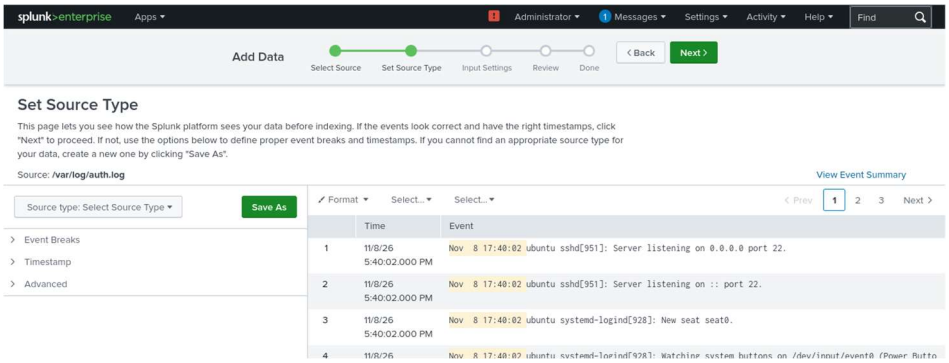


Figure 8. Set Source Type: a preview of auth.log events with timestamps.

Step 4: Verify the data arrived

I searched for the new source, and Splunk returned its events, each tagged with host ubuntu, source /var/log/auth.log and the source type I had chosen.

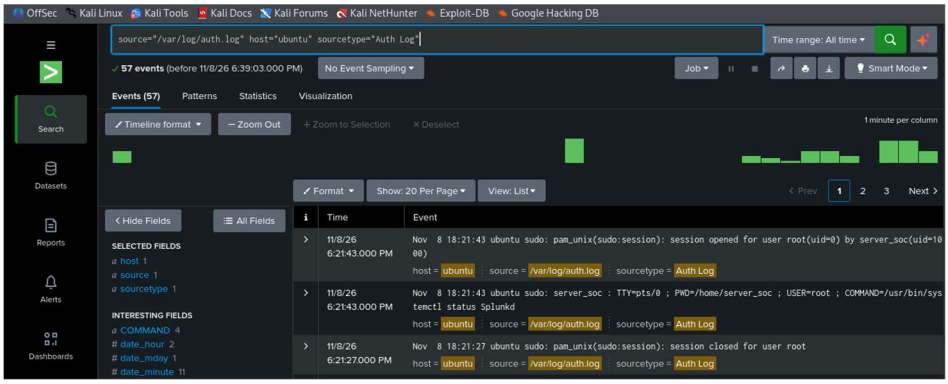


Figure 9. Search results for source /var/log/auth.log: 57 events.

A second source: /var/log/syslog

I repeated the process for the general system log. The review page confirms the settings: a file monitor, running continuously, with the source type syslog, host ubuntu and the default index.

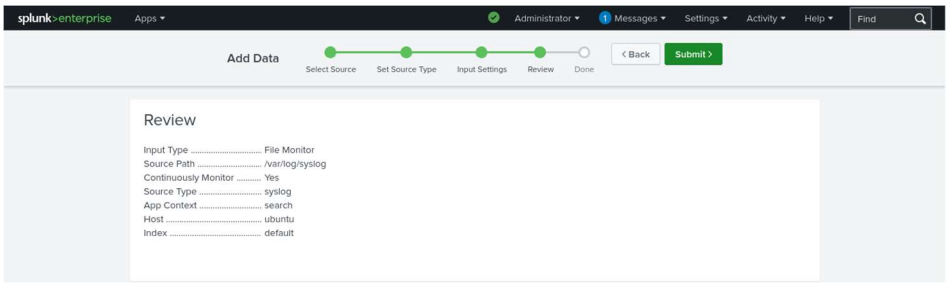


Figure 10. Review page for the syslog input.

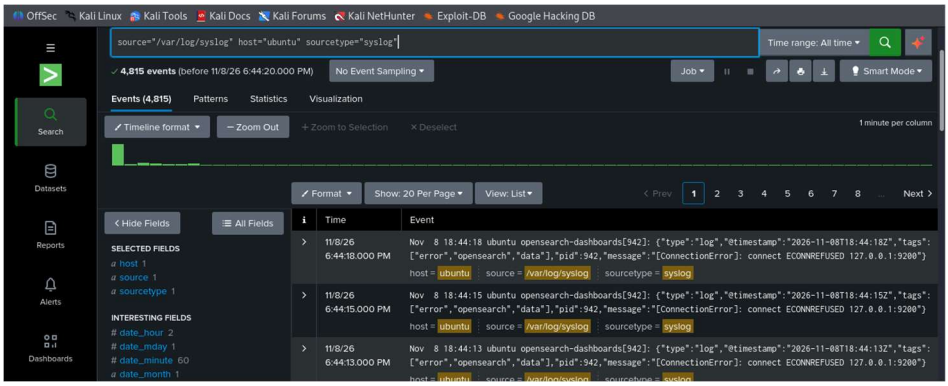


Figure 11. Search results for source /var/log/syslog: 4,815 events.

Source	Source type	Events found	What it holds
`/var/log/auth.log`	Auth Log	57	Logins, sudo use and SSH activity.
`/var/log/syslog`	syslog	4,815	General messages from the system and its services.

05 Creating Failed Logins to Find

To have something to search for, I generated failed SSH logins against my own virtual machine only. A short loop tries four usernames (admin, root, test, oracle), three times each, with a wrong password:

```
for u in admin root test oracle; do for i in 1 2 3; do
  sshpass -p 'wrongpass' ssh -o StrictHostKeyChecking=no -o PubkeyAuthentication=no
$u@127.0.0.1 true
done; done
```

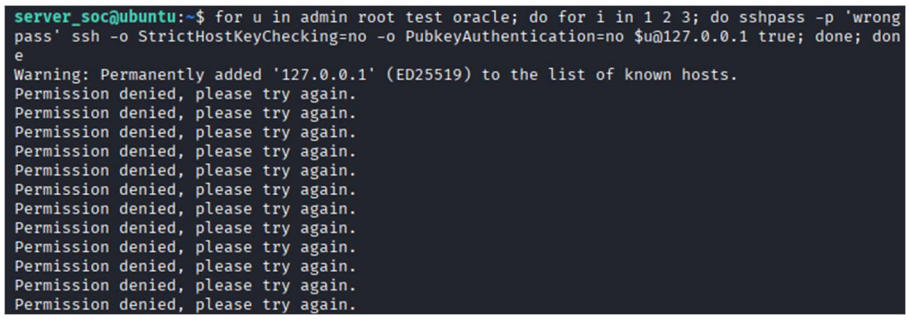


Figure 12. The loop: every attempt ends with "Permission denied".

I then checked the log. The failures are recorded; for example, sshd closed the connection of the invalid user oracle before authentication finished ("preauth"), as in my Week 2 work.

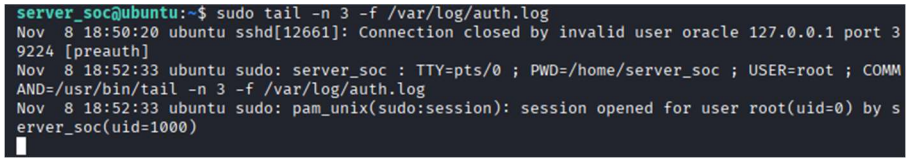


Figure 13. auth.log after the failed logins.

> **Safety**
> These attempts targeted 127.0.0.1, my own machine. Trying passwords against a system I do not own, or have no permission to test, is illegal.

06 The Searches

Search 1: view the failed logins

```
index=main "Failed password"
```

This looks in the main index for every event containing the phrase Failed password. Splunk returned 13 events, shown here from the newest. Each one reads "Failed password for invalid user ... from 127.0.0.1 port ... ssh2", with a different source port every time because each attempt opens a new connection. The timeline shows the events grouped in short bursts.

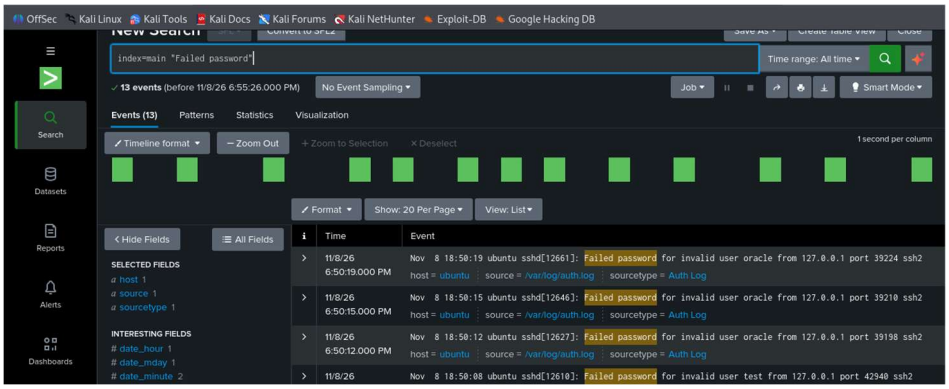


Figure 14. Search 1: 13 failed-password events.

Search 2: count them

```
index=main "Failed password" | stats count
```

The stats command turns the events into a number. The result is 13, matching the number of events in Search 1.

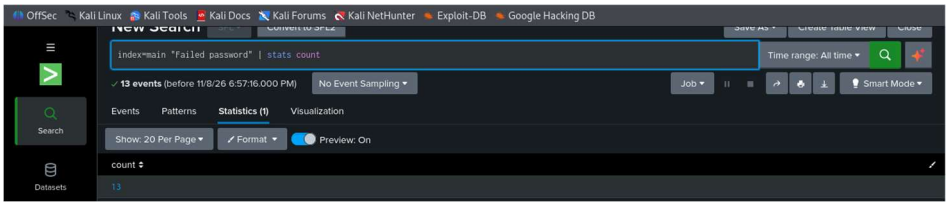


Figure 15. Search 2: the count is 13 (time range: All time).

The time-range filter

The time range picker, to the right of the search bar, restricts a search to a period such as the last 15 minutes, the last 24 hours or a custom date range. Searching All time returns every event ever indexed; narrowing the range shows only what happened in that window, which is how an analyst focuses on a suspected incident.

Search 3: who, from where, how often

```
index=main "Failed password" | rex "for (?:invalid user )?(?<user>\S+) from
(?<src_ip>\S+) port" | stats count by user, src_ip | sort - count
```

Part	What it does
`rex "..."`	Extracts two new fields from the text of each event with a regular expression: the username (user) and the source address (src_ip).
`stats count by user, src_ip`	Counts the events for each user and address pair.
`sort - count`	Orders the table with the highest count first.

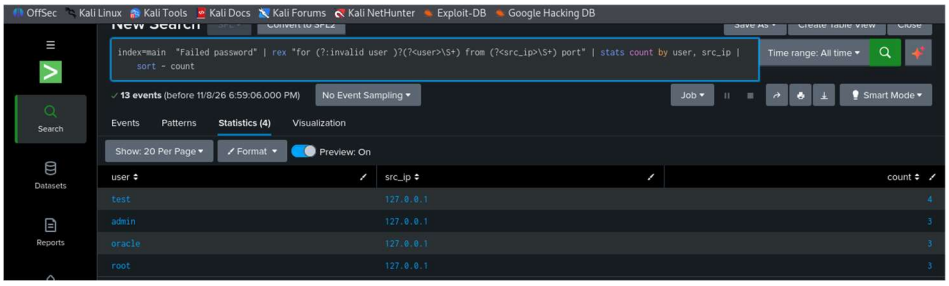


Figure 17. Search 3: failed logins by user and source address.

Bonus: failures over time

```
index=main "Failed password" | timechart span=1m count
```

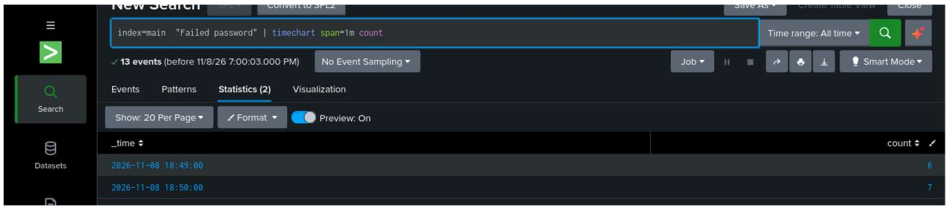


Figure 18. Failed logins per minute: 6 at 18:49 and 7 at 18:50.

07 What I Found

Question	Answer from the searches
How many failed logins?	13 events in total, confirmed by two different searches.
Which accounts?	test (4 attempts), and admin, oracle and root (3 attempts each). Most of them are invalid users, accounts that do not exist on the machine.
From where?	Only 127.0.0.1, the machine itself.
When?	Within about two minutes: 6 in the 18:49 minute and 7 in the 18:50 minute.
Check against my test	My loop made 12 attempts and Splunk shows 13. The extra attempt, for the user test, probably comes from an earlier manual try.

> **Analyst assessment**
> Thirteen failures across four different usernames within two minutes, from a single source, is exactly the shape of a brute-force or account-enumeration attack. Because the source is my own machine, it is a test and not an incident. On a real server the source would be an outside address that I would check against threat intelligence, and I would look for an "Accepted" line afterwards.

Compared with doing it by hand: in Week 2 I found failed logins with grep on one file. Splunk answered the same question across several sources, grouped the results by user and address, and drew them over time, all in a few seconds. It could also keep watching, and send an alert when a threshold is crossed.

08 SPL Cheat Sheet from This Task

Search or command	Use
`index=main "Failed password"`	Find events containing a phrase.
`| stats count`	Count the events.
`| stats count by field1, field2`	Count the events for each combination of fields.
`| rex "pattern"`	Extract new fields from text with a regular expression.
`| sort - count`	Sort the results, largest first.
`| timechart span=1m count`	Count events per time interval, for a chart.
`source="/var/log/auth.log"`	Limit a search to one source.

09 Reflection and Key Takeaways

- A SIEM is only as good as its data: permissions, correct source types and verifying that events arrive are the first steps.
- One precise phrase can still match too much. A loose search can pull in unrelated lines, so I check that the results are really what I expect.
- SPL makes the jump from "here are the events" to "here is what they mean" with only a few commands.
- The same failed-login pattern I analysed by hand in Week 2 became a count, a table and a timeline.

Next step: save these searches as a dashboard and create an alert for repeated failed logins.

10 References

- TryHackMe: Introduction to SIEM; Splunk documentation: getting data in and the Search Reference (SPL).